

Home Credit Project: Gradient Boosting

1. Feature Engineering

We start with the application data and add each data set each time to see whether the **ROC_AUC score** gets better.

AUC is the probability that if we randomly pick one default sample and one non-default sample, the model ranks the first as riskier.

In the project, we use AUC as the standard to evaluate the explanatory power of our model.

First, we set the baseline with Application data:
age, log-income..

then we add the other data sets by **HistGradientBoosting**, including:

mean, minimum, and maximum values,
counts of previous records,
sums of credit and debt,
late payments,
due days,
payment ratios,
credit utilization...

Pipeline: Median imputation, HistGradientBoostingClassifier, Five Fold Cross Validation

and **finally** got the results:

Set		Mean_ROC_AUC
0	Application	0.756787
1	+ Bureau	0.760437
2	+ Bureau Status	0.761416
3	+ Previous Application	0.763671
4	+ Installments	0.769354
5	+ Bureau Balance	0.769635
6	+ POS_CASH	0.775390
7	+ Credit Card Balance	0.778786

This suggests that adding information from customer's previous credit and payment histories improved the model's predictive performance.

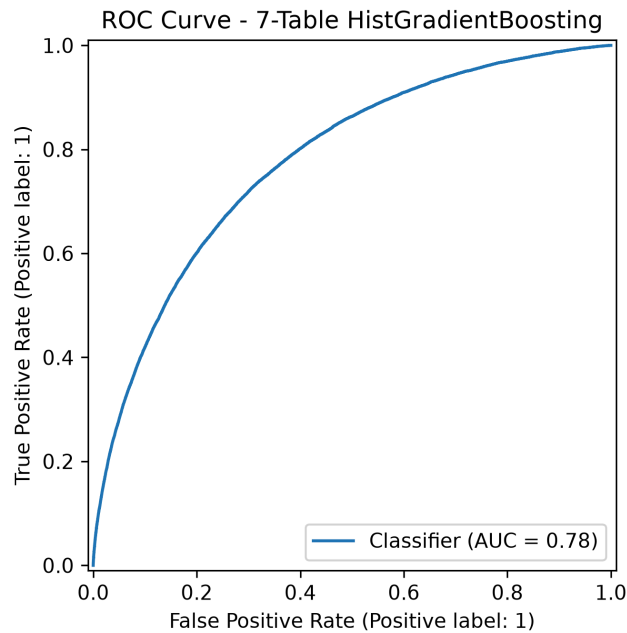
The final model uses features from all seven datasets and achieves a mean ROC-AUC of approximately 0.779 using 5-fold cross-validation.

Final ROC_AUC:

0.7773, 0.7836, 0.7765, 0.7816, 0.7750

For each iteration, we use four of them to train data and the last one to test. The process goes **five times** to cover all the data.

For the final model, we get the curves.

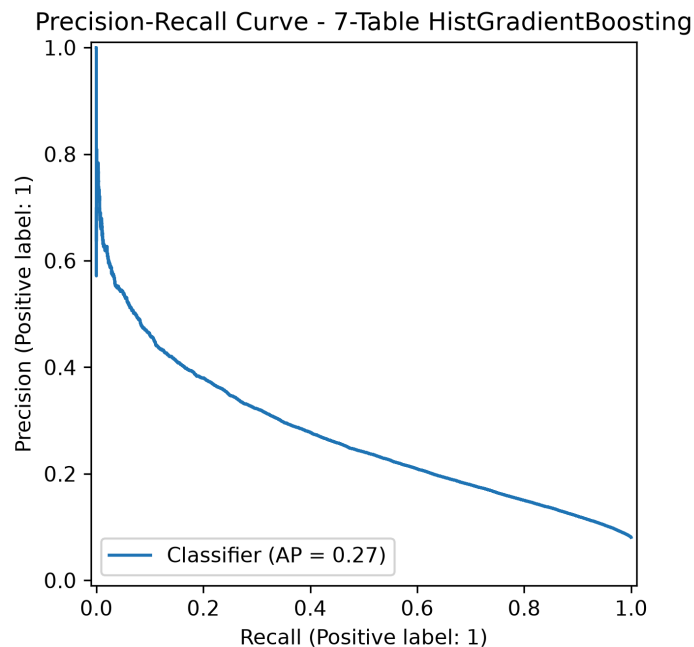


ROC Curve:

X-axis: False Positive Rate= % of good borrowers incorrectly marked as high-risk

Y-axis: True Positive Rate=% of actual defaulters caught

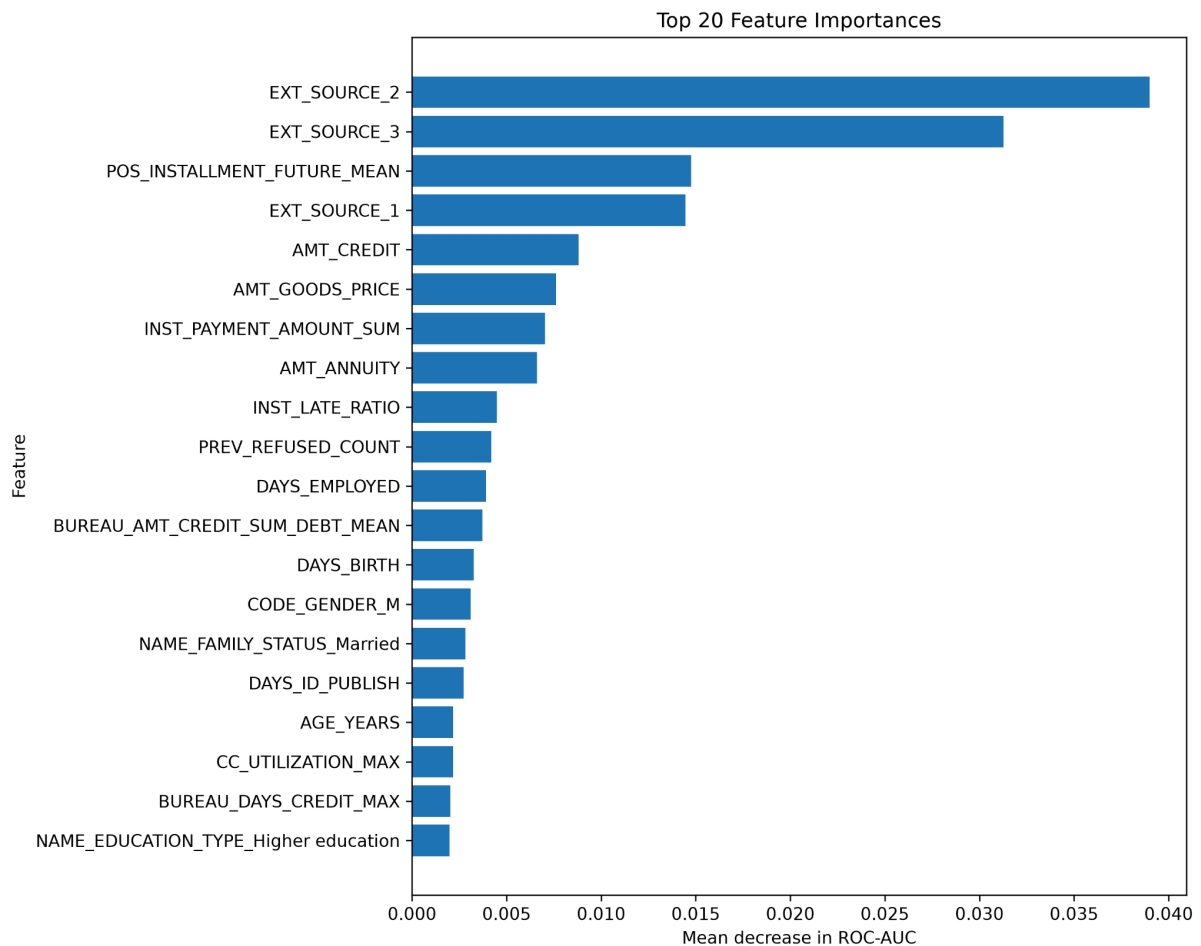
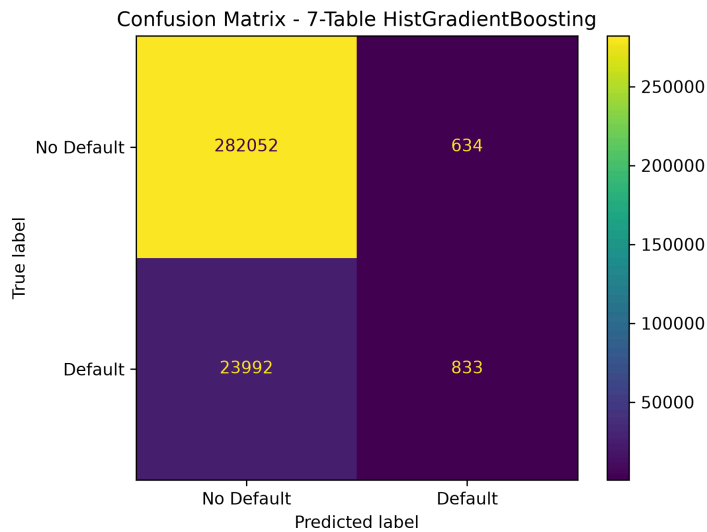
when AUC =0.5, this means a random guess, which is the diagonal line.



X-axis: Recall, sensitivity= $TP/TP+FN$, % of real defaulters the model catch

Y-axis: precision= $TP/TP+FR$, among all the high risk flagged people, what fraction actually default.

AP=0.27: when we label some high risk borrowers, most of them are still good borrowers. However, this is good because for Home Credit, we prefer getting good borrowers wrongly defined as high risk rather than treating high risk people as good ones.



2. Comparison between models

Here we just compared several machine learning models using the application data alone.

Logistic regression: 0.632

Random Forest: 0.737

HistGradientBoosting: 0.759

XGBoost:0.761

For current HistGradientBoosting model, we are using:

learning_rate:0.05

max_iteration:200

max_leaf_nodes=31

random_state=42

learning_rate:controls how much each new tree contributes to the overall model. Smaller learning rate means making smaller corrections at each step.

max_iter: controls the maximum number of boosting iterations, which is the maximum number of trees.

There will be a tradeoff between learning rate and max iteration. Taking a smaller step each time definitely gives a larger number of steps.

max_leaf_nodes: controls the maximum number of leaves in each individual decision tree. This is about the complexity of each tree. Increasing the parameter gives each tree more capacity to capture complicated patterns, but it can also increase the risk of overfitting.

random_state: controls the randomness used by the algorithm so that your results can be reproduced,